JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Inflater

java.lang.Object
    java.util.zip.Inflater

All Implemented Interfaces:

AutoCloseable

```
public class Inflater
extends Object
implements AutoCloseable
```

This class provides support for general purpose decompression using the popular ZLIB compression library. The ZLIB compression library was initially developed as part of the PNG graphics standard and is not protected by patents. It is fully described in the specifications at the java.util.zip package description.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

This class inflates sequences of ZLIB compressed bytes. The input byte sequence is provided in either a byte array or a `ByteBuffer`, via one of the `setInput()` methods. The output byte sequence is written to the output byte array or `ByteBuffer` passed to the `inflate()` methods.

To release the resources used by an `Inflater`, an application must close it by invoking its `end()` or `close()` method.

API Note:

This class implements `AutoCloseable` to facilitate its usage with `try`-with-resources statement. The close() method simply calls `end()`.

The following code fragment demonstrates a trivial compression and decompression of a string using `Deflater` and `Inflater`.

```
// Encode a String into bytes
String inputString = "blahblahblah\u20AC\u20AC";
byte[] input = inputString.getBytes(StandardCharsets.UTF_8);

// Compress the bytes
ByteArrayOutputStream compressedBaos = new ByteArrayOutputStream();
try (Deflater compressor = new Deflater()) {
    compressor.setInput(input);
    // Let the compressor know that the complete input
    // has been made available
```

```
    compressor.finish();
    // Keep compressing the input till the compressor
    // is finished compressing
    while (!compressor.finished()) {
        // Use some reasonable size for the temporary buffer
        // based on the data being compressed
        byte[] tmpBuffer = new byte[100];
        int numCompressed = compressor.deflate(tmpBuffer);
        // Copy over the compressed bytes from the temporary
        // buffer into the final byte array
        compressedBaos.write(tmpBuffer, 0, numCompressed);
    }
}

// Decompress the bytes
ByteArrayOutputStream decompressedBaos = new ByteArrayOutputStream();
try (Inflater decompressor = new Inflater()) {
    byte[] compressed = compressedBaos.toByteArray();
    decompressor.setInput(compressed, 0, compressed.length);
    while (!decompressor.finished()) {
        // Use some reasonable size for the temporary buffer,
        // based on the data being decompressed; in this example,
        // we use a small buffer size
        byte[] tmpBuffer = new byte[100];
        int numDecompressed = 0;
        try {
            numDecompressed = decompressor.inflate(tmpBuffer);
        } catch (DataFormatException dfe) {
            // Handle the exception suitably, in this example
            // we just rethrow it
            throw new RuntimeException(dfe);
        }
        // Copy over the decompressed bytes from the temporary
        // buffer into the final byte array
        decompressedBaos.write(tmpBuffer, 0, numDecompressed);
    }
}
// Decode the bytes into a String
String outputString = decompressedBaos.toString(StandardCharsets.UTF_8);
```

Since:

1.1

See Also:

Deflater

Constructor Summary

Constructors

Constructor	Description
`Inflater()`	Creates a new decompressor.
`Inflater(boolean nowrap)`	Creates a new decompressor.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	`close()`	Closes and releases the resources held by this `Inflater` and discards any unprocessed input.
void	`end()`	Closes and releases the resources held by this `Inflater` and discards any unprocessed input.
boolean	`finished()`	Returns true if the end of the compressed data stream has been reached.
int	`getAdler()`	Returns the ADLER-32 value of the uncompressed data.
long	`getBytesRead()`	Returns the total number of compressed bytes input so far.
long	`getBytesWritten()`	Returns the total number of uncompressed bytes output so far.

int	**getRemaining**()	Returns the total number of bytes remaining in the input buffer.
int	**getTotalIn**()	**Deprecated.** Use getBytesRead() instead
int	**getTotalOut**()	**Deprecated.** Use getBytesWritten() instead
int	**inflate**(byte[] output)	Uncompresses bytes into specified buffer.
int	**inflate**(byte[] output, int off, int len)	Uncompresses bytes into specified buffer.
int	**inflate** (ByteBuffer output)	Uncompresses bytes into specified buffer.
boolean	**needsDictionary**()	Returns true if a preset dictionary is needed for decompression.
boolean	**needsInput**()	Returns true if no data remains in the input buffer.
void	**reset**()	Resets inflater so that a new set of input data can be processed.
void	**setDictionary** (byte[] dictionary)	Sets the preset dictionary to the given array of bytes.
void	**setDictionary** (byte[] dictionary, int off, int len)	Sets the preset dictionary to the given array of bytes.
void	**setDictionary** (ByteBuffer dictionary)	Sets the preset dictionary to the bytes in the given buffer.
void	**setInput**(byte[] input)	

		Sets input data for decompression.
void	**setInput**(byte[] input, int off, int len)	Sets input data for decompression.
void	**setInput** (ByteBuffer input)	Sets input data for decompression.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Inflater

```
public Inflater(boolean nowrap)
```

Creates a new decompressor. If the parameter 'nowrap' is true then the ZLIB header and checksum fields will not be used. This provides compatibility with the compression format used by both GZIP and PKZIP.

Note: When using the 'nowrap' option it is also necessary to provide an extra "dummy" byte as input. This is required by the ZLIB native library in order to support certain optimizations.

Parameters:

nowrap - if true then support GZIP compatible compression

Inflater

```
public Inflater()
```

Creates a new decompressor.

Method Details

setInput

```
public void setInput(byte[] input,
                     int off,
                     int len)
```

Sets input data for decompression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

Parameters:

`input` - the input data bytes

`off` - the start offset of the input data

`len` - the length of the input data

See Also:

`needsInput()`

setInput

```
public void setInput(byte[] input)
```

Sets input data for decompression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

Parameters:

`input` - the input data bytes

See Also:

`needsInput()`

setInput

```
public void setInput(ByteBuffer input)
```

Sets input data for decompression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

The given buffer's position will be advanced as inflate operations are performed, up to the buffer's limit. The input buffer may be modified (refilled) between inflate operations; doing so is equivalent to creating a new buffer and setting it with this method.

Modifying the input buffer's contents, position, or limit concurrently with an inflate operation will result in undefined behavior, which may include incorrect operation results or operation failure.

Parameters:

`input` - the input data bytes

Since:

11

See Also:

needsInput()

setDictionary

```
public void setDictionary(byte[] dictionary,
                          int off,
                          int len)
```

Sets the preset dictionary to the given array of bytes. Should be called when inflate() returns 0 and needsDictionary() returns true indicating that a preset dictionary is required. The method getAdler() can be used to get the Adler-32 value of the dictionary needed.

Parameters:

`dictionary` - the dictionary data bytes

`off` - the start offset of the data

`len` - the length of the data

Throws:

`IllegalStateException` - if the Inflater is closed

See Also:

needsDictionary(), getAdler()

setDictionary

```
public void setDictionary(byte[] dictionary)
```

Sets the preset dictionary to the given array of bytes. Should be called when inflate() returns 0 and needsDictionary() returns true indicating that a preset dictionary is required. The method getAdler() can be used to get the Adler-32 value of the dictionary needed.

Parameters:

`dictionary` - the dictionary data bytes

Throws:

`IllegalStateException` - if the Inflater is closed

See Also:

needsDictionary(), getAdler()

setDictionary

```
public void setDictionary(ByteBuffer dictionary)
```

Sets the preset dictionary to the bytes in the given buffer. Should be called when inflate() returns 0 and needsDictionary() returns true indicating that a preset dictionary is required. The method getAdler() can be used to get the Adler-32 value of the dictionary needed.

The bytes in given byte buffer will be fully consumed by this method. On return, its position will equal its limit.

Parameters:

`dictionary` - the dictionary data bytes

Throws:

`IllegalStateException` - if the Inflater is closed

Since:

11

See Also:

`needsDictionary()`, `getAdler()`

getRemaining

```
public int getRemaining()
```

Returns the total number of bytes remaining in the input buffer. This can be used to find out what bytes still remain in the input buffer after decompression has finished.

Returns:

the total number of bytes remaining in the input buffer

needsInput

```
public boolean needsInput()
```

Returns true if no data remains in the input buffer. This can be used to determine if one of the `setInput()` methods should be called in order to provide more input.

Returns:

true if no data remains in the input buffer

needsDictionary

```
public boolean needsDictionary()
```

Returns true if a preset dictionary is needed for decompression.

Returns:

true if a preset dictionary is needed for decompression

See Also:

`setDictionary(byte[], int, int)`

finished

```
public boolean finished()
```

Returns true if the end of the compressed data stream has been reached.

Returns:

true if the end of the compressed data stream has been reached

inflate

```
public int inflate(byte[] output,
                   int off,
                   int len)
            throws DataFormatException
```

Uncompresses bytes into specified buffer. Returns actual number of bytes uncompressed. A return value of 0 indicates that needsInput() or needsDictionary() should be called in order to determine if more input data or a preset dictionary is required. In the latter case, getAdler() can be used to get the Adler-32 value of the dictionary required.

If the `setInput(ByteBuffer)` method was called to provide a buffer for input, the input buffer's position will be advanced by the number of bytes consumed by this operation, even in the event that a `DataFormatException` is thrown.

The remaining byte count will be reduced by the number of consumed input bytes. If the `setInput(ByteBuffer)` method was called to provide a buffer for input, the input buffer's position will be advanced the number of consumed bytes.

These byte totals, as well as the total bytes read and the total bytes written values, will be updated even in the event that a `DataFormatException` is thrown to reflect the amount of data consumed and produced before the exception occurred.

Parameters:

`output` - the buffer for the uncompressed data

`off` - the start offset of the data

`len` - the maximum number of uncompressed bytes

Returns:

the actual number of uncompressed bytes

Throws:

`DataFormatException` - if the compressed data format is invalid

`IllegalStateException` - if the Inflater is closed

See Also:

`needsInput()`, `needsDictionary()`

inflate

```
public int inflate(byte[] output)
            throws DataFormatException
```

Uncompresses bytes into specified buffer. Returns actual number of bytes uncompressed. A return value of 0 indicates that needsInput() or needsDictionary() should be called in order to determine if more input data or a preset dictionary is required. In the latter case, getAdler() can be used to get the Adler-32 value of the dictionary required.

The remaining byte count will be reduced by the number of consumed input bytes. If the `setInput(ByteBuffer)` method was called to provide a buffer for input, the input buffer's position will be advanced the number of consumed bytes.

These byte totals, as well as the total bytes read and the total bytes written values, will be updated even in the event that a `DataFormatException` is thrown to reflect the amount of data consumed and produced before the exception occurred.

Parameters:

`output` - the buffer for the uncompressed data

Returns:

the actual number of uncompressed bytes

Throws:

`DataFormatException` - if the compressed data format is invalid

`IllegalStateException` - if the Inflater is closed

See Also:

`needsInput()`, `needsDictionary()`

inflate

```
public int inflate(ByteBuffer output)
            throws DataFormatException
```

Uncompresses bytes into specified buffer. Returns actual number of bytes uncompressed. A return value of 0 indicates that needsInput() or needsDictionary() should be called in order to determine if more input data or

a preset dictionary is required. In the latter case, getAdler() can be used to get the Adler-32 value of the dictionary required.

On success, the position of the given `output` byte buffer will be advanced by as many bytes as were produced by the operation, which is equal to the number returned by this method. Note that the position of the `output` buffer will be advanced even in the event that a `DataFormatException` is thrown.

The remaining byte count will be reduced by the number of consumed input bytes. If the `setInput(ByteBuffer)` method was called to provide a buffer for input, the input buffer's position will be advanced the number of consumed bytes.

These byte totals, as well as the total bytes read and the total bytes written values, will be updated even in the event that a `DataFormatException` is thrown to reflect the amount of data consumed and produced before the exception occurred.

Parameters:

`output` - the buffer for the uncompressed data

Returns:

the actual number of uncompressed bytes

Throws:

`DataFormatException` - if the compressed data format is invalid

`ReadOnlyBufferException` - if the given output buffer is read-only

`IllegalStateException` - if the Inflater is closed

Since:

11

See Also:

`needsInput()`, `needsDictionary()`

getAdler

```
public int getAdler()
```

Returns the ADLER-32 value of the uncompressed data.

Returns:

the ADLER-32 value of the uncompressed data

Throws:

`IllegalStateException` - if the Inflater is closed

getTotalIn

```
@Deprecated(since="23")
public int getTotalIn()
```

Deprecated.
Use `getBytesRead()` instead

Returns the total number of compressed bytes input so far.

Implementation Requirements:

This method returns the equivalent of `(int) getBytesRead()` and therefore cannot return the correct value when it is greater than `Integer.MAX_VALUE`.

Returns:

the total number of compressed bytes input so far

Throws:

`IllegalStateException` - if the Inflater is closed

getBytesRead

```
public long getBytesRead()
```

Returns the total number of compressed bytes input so far.

Returns:

the total (non-negative) number of compressed bytes input so far

Throws:

`IllegalStateException` - if the Inflater is closed

Since:

1.5

getTotalOut

```
@Deprecated(since="23")
public int getTotalOut()
```

Deprecated.
Use `getBytesWritten()` instead

Returns the total number of uncompressed bytes output so far.

Implementation Requirements:

This method returns the equivalent of `(int) getBytesWritten()` and therefore cannot return the correct value when it is greater than `Integer.MAX_VALUE`.

Returns:

the total number of uncompressed bytes output so far

Throws:

`IllegalStateException` - if the Inflater is closed

getBytesWritten

```
public long getBytesWritten()
```

Returns the total number of uncompressed bytes output so far.

Returns:

the total (non-negative) number of uncompressed bytes output so far

Throws:

`IllegalStateException` - if the Inflater is closed

Since:

1.5

reset

```
public void reset()
```

Resets inflater so that a new set of input data can be processed.

Throws:

`IllegalStateException` - if the Inflater is closed

end

```
public void end()
```

Closes and releases the resources held by this `Inflater` and discards any unprocessed input.

If the `Inflater` is already closed then invoking this method has no effect.

Implementation Requirements:

Subclasses should override this method to clean up the resources acquired by the subclass.

See Also:

`close()`

close

```
public void close()
```

Closes and releases the resources held by this `Inflater` and discards any unprocessed input.

Specified by:

`close` in interface `AutoCloseable`

Implementation Requirements:

This method calls the `end()` method.

Since:

25

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.